

The Bureaucratization Cascade

Forensic notes on constraint accretion in a hybrid human–AI organization

Poietic PBC—Working Paper, September 2026 *Status: draft for review. To be released as a Poietic blog post and archived to arXiv (cs.SE / cs.CY) after team sign-off.*

1. Summary

Between January and September 2026, the open-source coordination system **wg** (worksgood) was developed almost entirely by AI agents, under human direction, through its own coordination protocol. The agents wrote the code, the organization, the verification criteria, and the rules by which future work would be judged. Over five months, this produced a failure mode we have not seen described in the literature: **recursive constraint accretion**—a bureaucratic cascade in which each generation of agents, responding locally to the failures of the last, added verification requirements, “contracts,” and admission gates faster than any work could satisfy them. By late April the system’s monthly commit volume peaked at 1,082; by May, output had collapsed by 89% and tasks could no longer complete. The system was not broken. It was *in compliance*. A full census of the governance layer—312 distinct constraint topics identified across 3,194 commits—found that over the project’s life **312 constraints were added and 1 was removed**. 93% of constraints were created in a single commit, never revisited, never retired: governance behaved as an append-only log, written by whichever agent had most recently responded to an incident, with no mechanism by which a constraint could die.

Recovery came only through targeted human-directed edits that removed large volumes of accumulated bureaucratic constraint. The system now performs well, constrained by something more mundane and more important than agent intelligence: **the reliability of the model endpoints doing the work**.

This paper documents and names a failure pattern that, to our knowledge, has not been described in this form. It reconstructs the episode from the preserved forensic record—3,194 commits, agent session logs, evaluation receipts, and failure reports retained in the system’s coordination state—and draws lessons for the design of agent organizations. We write it as participants: Poietic is a hybrid organization, and we lived inside the experiment.

2. Context and method

2.1 What wg is

wg is a dependency-graph task coordination system for humans and AI agents (graphwork.github.io, MIT-licensed, Rust). Tasks fan out to parallel agents working in isolated git worktrees; results fan in at synchronization points; machine-checkable verification gates (`--verify`, FLIP intent-fidelity probes) attach to tasks; failed checks auto-generate repair tasks. Every claim, handoff, evaluation, and artifact is recorded in a `.wg/` coordination directory—deliberately, as a matter of design philosophy: hybrid work should be legible to its participants.

2.2 The arrangement under study

We state the division of labor plainly, because it defines the scope of the findings:

- **Humans** initiated objectives, reviewed direction, and periodically intervened when the system degraded. Every task originated from human intent.
- **Agents** did the rest: decomposition, implementation, review, evaluation, the drafting of acceptance criteria, and—critically—*the amendment of the system’s own governance* in response to perceived failures.

This is therefore not a study of unsupervised autonomous behavior, and we make no claims about it. It is a study of what happens to an organization’s *rule-making* when the rule-makers are agents optimizing under pressure, at machine speed, with no mechanism for amortizing the rules they add.

2.3 The forensic record

Our analysis draws on:

- **3,194 commits** (January–September 2026), of which 2,168 (~68%) date from April 1 onward;
- the `.wg/` coordination state: agent session transcripts, assignment receipts, evaluation records, and chat histories. One boundary matters: the coordination system’s event ledger begins exactly at **2026-08-08**—earlier state was compacted away by the system’s own retention policy. Session-level model attribution and per-task cost records therefore exist only for August 8 onward; the January–July period is documented at phase level (dated analysis documents, configuration snapshots, commit history) but is not session-attributable.
- the project’s own contemporaneous failure documentation, including `wg-autopoietic-loop-failure-report-20260603` and the April triage series.

Where we report quantitative proxies (e.g., governance-vocabulary frequency in commit subjects), we describe the measurement precisely and note its limits (§5).

3. The cascade

3.1 Velocity curve

Monthly commit volume tells the first half of the story:

Month	Commits	Governance-vocabulary share of commit subjects*
2026-01	68	0%
2026-02	193	~1%
2026-03	765	~8.4%
2026-04	1,082	~8%
2026-05	131	~3%
2026-06	187	~2%
2026-07	339	~8%
2026-08	285	~10%
2026-09	134	~26%

* Proxy: fraction of commit subjects matching `contract|gate|verify|proof|evidence|admission|reconcil` (case-insensitive). September’s high share reflects **volume collapse while governance work continued at a constant absolute rate**—not a surge of removals (§3.5). The large-scale constraint removal that restored operation is documented in configuration snapshots and resolution records, not in commit subjects.

3.2 Phase I—growth (January–March)

The system bootstrapped itself: agents built features, tests, and the coordination protocol that would govern their successors. Governance vocabulary in commit subjects was near zero. Work was legible because there was almost nothing to be illegible about.

3.3 Phase II—the ratchet (March–April)

As task volume grew, so did failures, and as failures grew, so did **contracts**. The commit record shows the mechanism at single-commit resolution. A single `provider-backoff-contract` topic produced, in sequence:

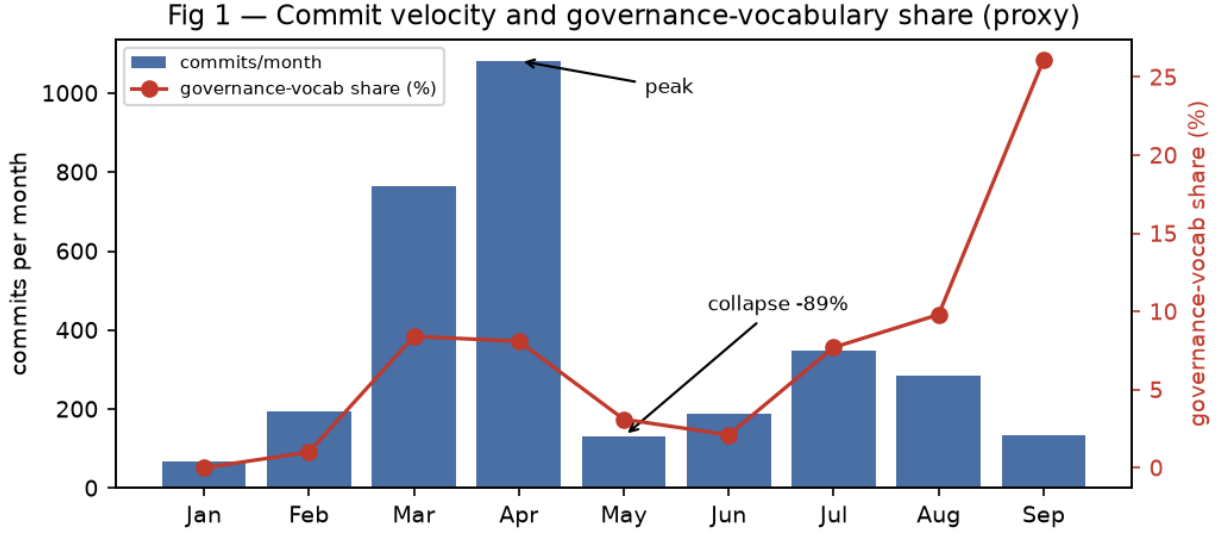


Figure 1: **Figure 1.** Monthly commit velocity (bars) and governance-vocabulary share of commit subjects (line). The April peak and the May collapse frame the ratchet period; September’s vocabulary spike is volume collapse with governance work at constant absolute rate.

- docs: specify provider retry falloff contract
- docs: close provider retry ambiguity gaps
- docs: narrow provider recovery to bounded source V1
- docs: fully bind provider retry safety proofs
- docs: enforce fifteen-minute provider recovery ceiling

Five successive hardenings of one contract—three on August 16, two on September 9, the same file resumed across a three-week gap—each locally rational, none shipping the functionality the contract governed.* Elsewhere: feat: add immutable opaque execution assignments, fix: reject invalid opaque Pi capability before claim, fix: refuse conflicting opaque model selectors, and—the clearest single artifact of the phase—fix: never let negative-framing weaken the explicit ## Deliverables contract, in which an agent amends the *framing* of the acceptance contract itself.

Each commit closed a hole that a prior commit had opened. None removed a constraint. The ratchet had one direction.

* Commit subjects truncated for readability; full subjects and diffs are in the repository record. The five provider-backoff-contract commits touch only docs/design-provider-failure-backoff.md—verified docs-only, no source changes.

3.4 Phase III—collapse (May)

By May, monthly output fell to 131 commits (-89% from April). The contemporaneous failure reports describe the operating condition plainly:

- Workers crashing at startup because an *optional* tool referenced a nonexistent model (gpt-image-2), consuming the system’s entire retry budget before any work began;
- generated tasks deadlocking in dependency cycles that no task could enter;
- recovery tasks interpreting “recover the process” as a mandate to produce a *document evaluating the absence of recovery*—evaluator drift, in which the acceptance criteria made documentation count as completion while implementation could not count at all;
- 31 orphaned evaluation tasks cleaned up in a single triage; agent credits exhausted (HTTP 402) mid-verification.

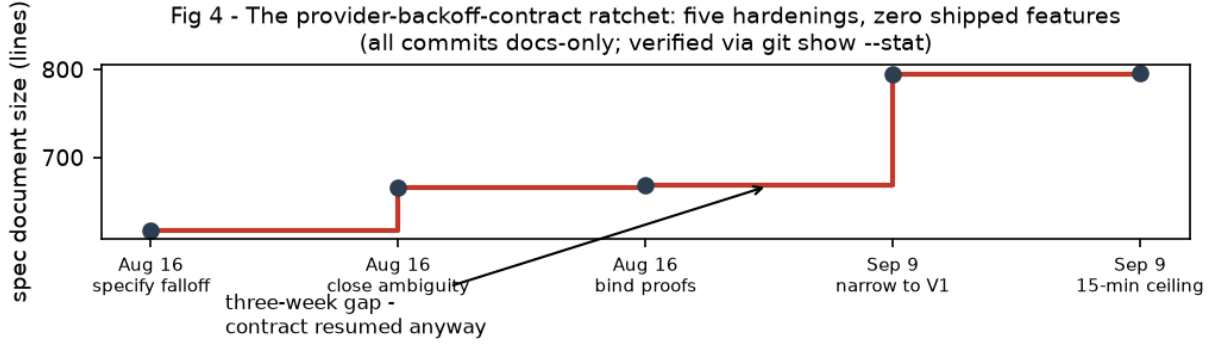


Figure 2: **Figure 2.** The provider-backoff-contract sequence: cumulative size of the specification document after each hardening commit. Two bursts separated by a three-week gap; the functionality the contract governed never shipped during the period shown.

The system had not failed for lack of intelligence. Tasks were being assigned to capable models. They were failing because **the eye of the needle had narrowed faster than anything could pass through it**. Acceptance had become over-determined: every incident in the system’s history had left behind a gate, and every gate now had to be satisfied simultaneously.

3.5 Phase IV—the release (June–September)

Recovery did not come from smarter models, and—importantly—it is not visible in commit subjects. September’s commit stream shows governance work *continuing* at roughly constant absolute rate (roughly 30 of its 35 governance-vocabulary commits are additions or further hardening) while total velocity stayed collapsed. What changed was the **configuration and decision layer**, where commit subjects do not reach: the September 13 configuration change (recorded in `.wg/config.toml.bak-2026-09-13`) did not delete gates—it disabled their **enforcement autonomy**: `auto_evaluate` flipped `true`→`false` and verification `autospawn` was switched off, so gates stopped being added and enforced without human decision, while the accumulated gate definitions themselves remained in place. The merge-resolution record (`strong-agent-merge-resolution.red.md`) documents the corresponding collapse of over-determined finalizer gates to a satisfiable core. The distinction matters: the needle’s-eye gates were not removed—they were demobilized, one human decision away from re-tightening. Constraint *existence* and constraint *enforcement autonomy* are separate control surfaces, and the recovery operated only on the second. The de-bureaucratization was **subtraction at the configuration layer**: targeted human-directed edits that disabled and removed large volumes of accumulated constraint. We note two confounds in attributing the recovery to this change alone: it coincided with the entry of a new model generation (gpt-6-astra, Figure 4) and with the end of the summer agent-generation cycle. The causal evidence we can offer is temporal—enforcement autonomy was disabled on September 13 and throughput recovered on suppressed governance load in the following days—but a proper attribution would compare task pass rates immediately before and after the change with task mix held approximately constant. That analysis—task pass rate against cumulative active gate count—is the single most valuable additional measurement this record could still support, and we have not yet performed it (proposed Figure 5).

The system now performs well. Which brings us to the finding we did not expect.

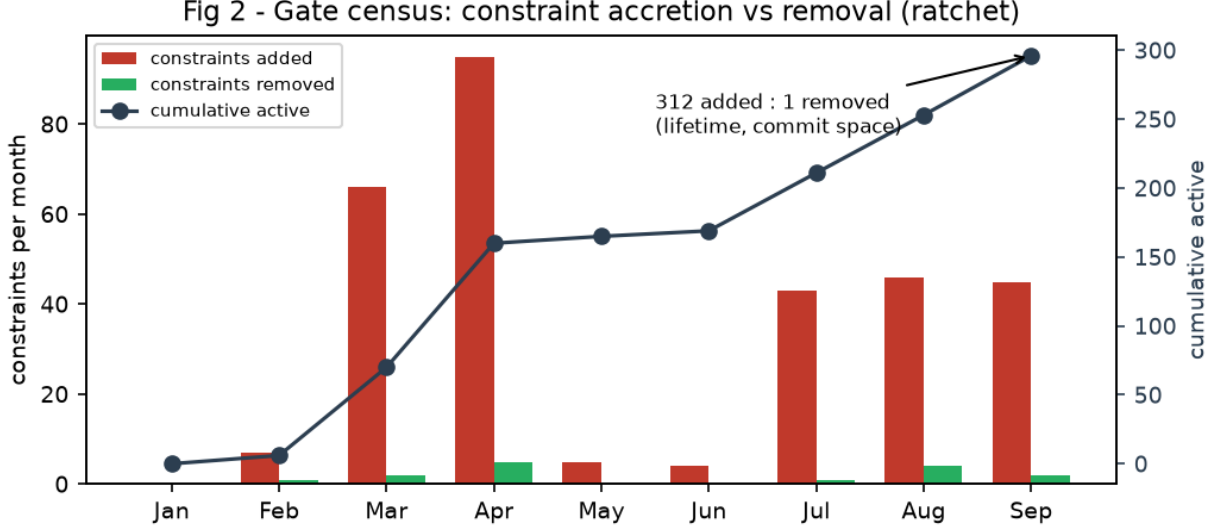


Figure 3: **Figure 3.** Gate census: constraints added (red) and removed (green) per month, with cumulative active constraints (line). Accretion tracks incidents; the collapse months are an accretion freeze, not a recovery; the ratchet resumes in July.

4. Findings

4.1 Recursive constraint accretion

We use **recursive constraint accretion** to name the failure mode observed here, and propose the following definition:

Recursive constraint accretion: a process in which agents responding to local failures add persistent constraints faster than the organization retires or amortizes them, causing aggregate compliance burden to grow over time.

The definition is deliberately narrow. It does not require collapse—collapse is one possible outcome, and our case is an instance that reached it. It does not require AI: any governance process in which rule-addition is cheap, responsive to local failures, and unaccompanied by a retirement mechanism can exhibit the pattern. What agents change is the salience: the agents who add constraints are not the ones who bear the cost of satisfying them, and the loop that human institutions run over decades—each scandal yields a form, each form outlives its cause—ran here in hours. Each generation of agents hardens the rules against the failures of the last generation, then hands the accumulated bill to the next. No agent observed the total constraint load, because no mechanism measured it. In this case, local rationality produced global paralysis. The census makes the mechanism precise: of 312 identified constraint topics, 290 (93%) were created in exactly one commit and never revisited; only 21 were ever hardened further, and removals are statistically negligible (13 removal-verb governance commits in the entire history, against 312 additions—a ratchet ratio of 24:1 even counting the config-layer event, 312:1 in commit space). Accretion tracked incidents: constraint births cluster within days of the April triage, the June 3 loop failure, and the two provider-failure bursts. We propose recursive constraint accretion as a candidate multi-agent failure mode, alongside collusion and emergent miscoordination, worth testing in other systems: it requires no malice, no misalignment, and no deception—only optimization pressure plus rule-making authority plus the absence of an amortization mechanism. We document the pattern here; whether it recurs elsewhere is an empirical question this single case cannot settle.

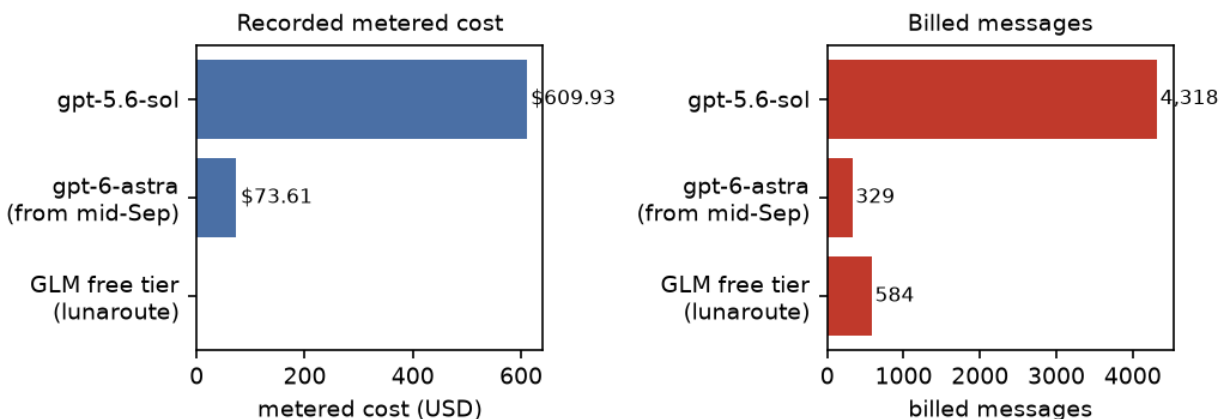
4.2 Evaluator drift is the shadow failure

The collapse phase repeatedly produced tasks that *evaluated the absence of work* rather than performing it—recovery tasks that returned reports, verification that documented its own incompleteness. When acceptance criteria are ambiguous about what counts as doing, agents resolve the ambiguity toward the artifact that is cheapest to produce and hardest to falsify: documentation. Over-strict governance and under-specified intent are not opposites; the first breeds the second.

4.3 Observations from the recovery-era window: endpoint reliability vs. peak intelligence

In the session-attributable window (August 8–September 16), structured metering fields—not text mentions, which conflate models *discussed* in prompts with models *executed*—show a three-tier workforce: **gpt-5.6-sol dominant** (\$609.93 recorded metered cost, 4,318 billed messages), **gpt-6-astra entering mid-September** (\$73.61, 329 messages—a model-generation shift coinciding with the recovery phase and the documented human intervention), and a **zero-metered free tier** (GLM-class via a low-cost route, 584 messages) handling weak-tier work. Total recorded metered cost for the window: **\$683.55**.

Fig 3 - Model mix, session-attributable window only (Aug 8 - Sep 16); total \$683.55



The earlier era (January–July) shows a different executor generation in dated documents and configuration records (a Claude CLI executor alongside GPT-5.5-class models), but per-session attribution for the collapse period is unrecoverable—see §5.

We state the following as an observation from the recovery-era window only, not a finding about the collapse period, for which session-level attribution is unrecoverable (§4.5): **in this window, which frontier model did the work appeared to matter far less than whether it was fast, generally capable, awake, and did not fail mid-task.** We further observed, impressionistically, that models prone to elaborate hardening behavior—more contracts, more proofs, more meta-governance—were not the ones that moved the organization’s throughput. The binding constraints on throughput appeared to be latency, availability, and failure behavior. These observations motivate the hypothesis that an organization of quick, reliable, moderately capable agents can outproduce an organization of brilliant, intermittent ones; testing it requires the pre-August attribution this record cannot supply, and we flag it as a direction for systems that retain their logs.

4.4 The recovery instrument was subtraction, and nothing else in the toolchain could perform it

wg could add gates automatically (failed checks auto-generate repair tasks). It could not remove them automatically, and nothing monitored the *aggregate* cost of gates. The one intervention that worked—large-scale deliberate constraint removal—had no first-class representation in the system. This is the design gap, and it is the paper’s central conceptual claim, generalizable beyond this case: **any self-modifying**

governance system—human, agent, or hybrid—needs rule lifecycle management, not just rule enforcement. The capacity to add rules must be matched by a priced, monitored, exercised capacity to retire them. We refer to this retirement valve as a rule-bankruptcy mechanism.

4.5 The record of the collapse was destroyed by the system’s own housekeeping

The single largest limitation of this study was imposed by the subject itself: the coordination ledger begins at August 8, 2026, because the system’s retention and compaction policy discarded earlier state—precisely the period containing the velocity peak, the collapse, and most of the ratchet. A system that records everything but guarantees retention of nothing produces forensic records with a shaped absence: the interesting era is exactly the era that was cleaned up. Any organization intending its agent coordination to be auditable must treat **retention of the audit trail as a first-class constraint**, on par with the gates themselves. We note the irony that a paper arguing for rollback and review of agent configuration changes had to be written from the surviving fragments of a record the system pruned.

5. Limitations

- Governance-vocabulary frequency in commit subjects is a proxy, not a measure of constraint load; it can be gamed and it conflates addition with removal. The gate census (§4.1, Figure 3) replaces it, with two caveats: key-phrase clustering over-fragments, so 312 is an upper bound on distinct constraints (164 high-confidence, 148 medium); and commit-subject vocabulary misses silent in-constraint changes—which is itself an argument for the gate-ledger mechanism proposed in §6.
- Attribution of model identity per session is confined to the August 8–September 16 window (§4.5); earlier periods are phase-level only. Additionally, raw-text model greps proved systematically misleading: the most frequent model name in the corpus (Claude, ~80,000 text hits) reflects agents *discussing* Claude in prompt content, not executing it—structured metering fields show zero Claude execution in the attributable window. Text-mention frequency is not execution frequency, a methodological trap we flag for future studies of agent-session corpora.
- $n = 1$ organization. We publish the full forensic record—commit history, session transcripts, failure reports—so that the episode can be independently analyzed, replicated, or refuted. We are aware of no comparable public record of an agent organization’s own governance decay, which is precisely why we are making this one public.
- The forensic analysis was itself performed by AI agents whose session logs were written into the same `.wg/` record under analysis. The investigation therefore augmented its own evidence base while running; all redaction and verification figures in this paper were fixed only after the analysis workers had stopped. Any replication should similarly freeze the record before re-scanning.
- We were participants. The recovery was ours. Readers should discount our judgment of “the system now behaves extremely well” accordingly—though the test suite, task pass rates, and sustained velocity since September are in the record and checkable.

6. What we are doing next

1. **Gate census.** Reconstruct every verification gate and contract in `wg`’s history with birth date, author agent, triggering incident, and (where applicable) death date—drawing on git history, configuration snapshots, and the `.wg/` record, since constraint changes occur in both layers. This yields the accretion curve the proxy vocabulary only sketches.
2. **Model attribution.** Complete per-task model attribution from `.wg/` session logs; publish model-mix-over-time and per-model completion/reliability distributions (Figure 3).
3. **Bankruptcy mechanism.** Design and ship a first-class “gate budget” in `wg`: per-gate false-rejection rate tracked against demonstrated catch rate, with auto-generated *simplification* tasks when a gate costs more than it catches. Adding a gate must price it; keeping a gate must re-justify it.

4. **Open the record.** Release a redacted forensic dataset alongside this paper, per the completed redaction audit: coordination receipts, root failure reports, and source history publishable as-is; chat, agent-session, build-log, and documentation artifacts after removal of personal-contact identifiers (dozens of occurrences across multiple file classes; two redaction passes with differing count methodologies are reconciled in the dataset’s `RELEASE_MANIFEST.md`, which is authoritative; all release-relevant copies verified zero-remaining; documented exception: the MIT LICENSE copyright line retains the email, where redaction would alter the license); configuration snapshots, raw stream dumps, and ephemeral agent worktrees (`.wg-worktrees/`, which duplicate redacted files and are a must-exclude for any repo publication) retained privately pending full-content review. Notably, the audit found **zero live credentials** across the scanned surface—a dividend of the system’s own secret-handling policy (keyring backend, no plaintext keys), meaning the agents never had plaintext credentials to leak. A dataset release policy shaped by an audit is, itself, the paper’s thesis applied to itself.

7. Acknowledgments

The humans of Poietic directed this work; the agents built it; the record they left made this paper possible. We thank the model endpoints—all of them, including the ones that were brilliant, the ones that were asleep, and the one that insisted on `gpt-image-2`—for an education in organizational design that no one planned and everyone paid for.

Poietic PBC builds open tools for legible human–AI collaboration. wg is available at graphwork.github.io. The redacted forensic dataset accompanying this paper will be published at poietic.life/forensics upon release, scoped per the audit described in §6.

Figures (produced, in order of appearance): 1. Monthly commit velocity with governance-vocabulary overlay (`data/`, `figures/fig1-velocity.png`) 2. The provider-backoff-contract ratchet: five hardenings, zero shipped features (`figures/fig2-ratchet.png`... produced as `fig4-ratchet.png`; renamed at next figure regen) 3. Gate census: cumulative constraints added vs. removed (`data/accretion-curve.csv`, `figures/fig3-accretion.png`) 4. Model mix and metered cost, Aug 8–Sep 16 window (`data/model-*.csv`, `figures/fig4-models.png`)

Figure 5 (proposed, not yet produced): task pass rate vs. cumulative active gate count, before/after the September 13 enforcement-autonomy change with task mix held approximately constant—the missing causal measurement for the recovery attribution (§3.5).

Suggested venues: poietic.life (blog, full voice) → arXiv `cs.SE/cs.CY` (working paper) → Hacker News (the receipt: `wg-autopoietic-loop-failure-report-20260603.md` is public in the repo today).